Heart Disease Risk Prediction

End-to-End MLOps Pipeline

Machine Learning Operations (MLOps) - AIMLCZG523 - Assignment 01

Student	Dinesh Kumar Yadav
ID	2025CT05018
Programme	M.Tech (AI/ML), BITS Pilani WILP
Code repository	https://github.com/dinesh-github-2017/heart-disease-mlops
Pipeline video	link to your pipeline video, or file name if submitted separately
Date	16 October 2026

Contents

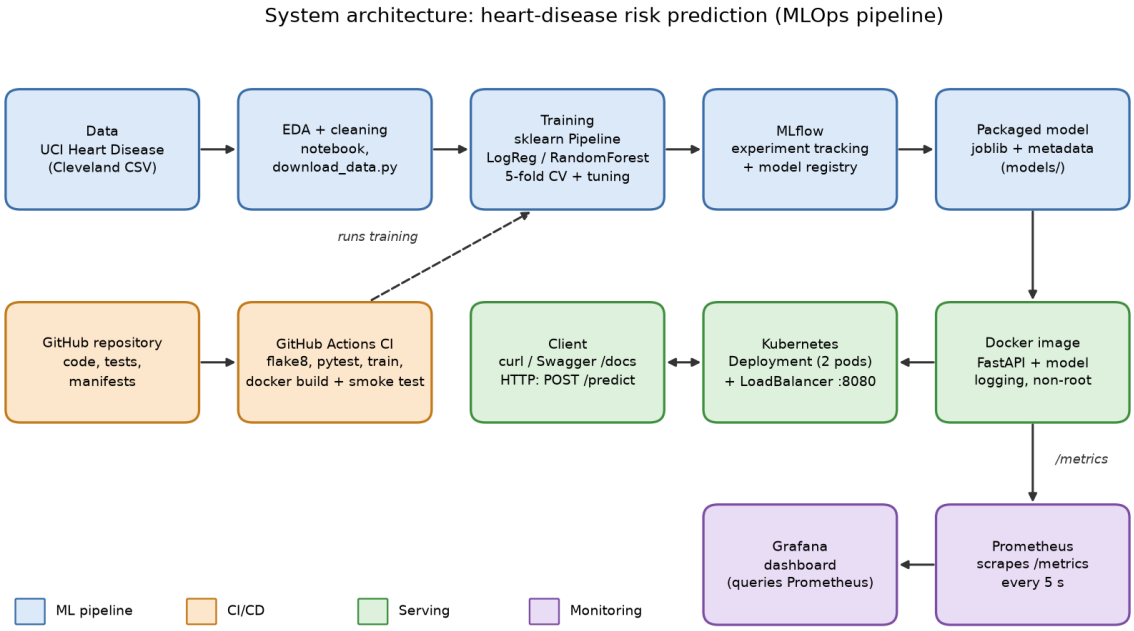
1. Introduction and Objective 2
2. Architecture 2
3. Setup and Installation 2
4. Data Acquisition and EDA 3
5. Feature Engineering and Model Development 4
6. Experiment Tracking with MLflow 5
7. Model Packaging and Reproducibility 6
8. CI/CD Pipeline and Automated Testing 6
9. Model Containerisation and API 7
10. Production Deployment on Kubernetes 8
11. Monitoring and Logging 8
12. Repository, Deliverables and Limitations 9

1. Introduction and Objective

The goal of this project is to build a classifier that predicts the risk of heart disease from patient health data and to deliver it as a reproducible, tested, containerised and monitored API, following MLOps practice. The data is the Cleveland subset of the UCI Heart Disease dataset (age, sex, chest pain type, blood pressure, cholesterol, ECG results, maximum heart rate and related measurements). The original 0-4 disease grade is converted to a binary target (0 = no disease, 1 = disease). The cleaned dataset has 303 patient records and 14 columns (13 features plus the target); 139 records (45.9%) are labelled as heart disease present and 164 (54.1%) as absent.

The project covers the full lifecycle: data acquisition and EDA, feature engineering, model training and tuning, experiment tracking with MLflow, packaging, automated testing and CI/CD with GitHub Actions, containerisation with Docker, deployment on Kubernetes, and monitoring with Prometheus and Grafana.

2. Architecture



Architecture of the pipeline (generated by scripts/build_report.py).

Data flows left to right through the ML pipeline into a packaged model. The model is baked into a Docker image with a FastAPI service, which is deployed on Kubernetes and exposed through a LoadBalancer service. GitHub Actions lints, tests, trains and builds the image on every push. The service exposes Prometheus metrics, which Grafana visualises.

3. Setup and Installation

Requirements: Python 3.11, Git, Docker Desktop (with Kubernetes enabled for Section 8).

```
git clone <repository-url>
cd heart-disease-mlops
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt -r requirements-dev.txt
python src/download_data.py          # download + clean the dataset
python -m src.train                  # train, track with MLflow, save the model
mlflow ui --backend-store-uri sqlite:///mlflow.db
python -m pytest -v                  # unit tests
flake8 src api tests                 # lint
docker build -t heart-disease-api:v1 .
docker run --rm -p 8000:8000 heart-disease-api:v1
kubectl apply -f k8s/                # Kubernetes deployment
docker compose up -d                 # API + Prometheus + Grafana
python scripts/generate_traffic.py --n 200
```

The repository layout is: **src/** (data, features, training, prediction), **api/** (FastAPI service), **tests/** (pytest), **k8s/** (manifests), **monitoring/** (Prometheus and Grafana configuration), **.github/workflows/** (CI), **notebooks/** (EDA) and **screenshots/**.

4. Data Acquisition and EDA

Acquisition. *src/download_data.py* downloads the raw data from the UCI repository, assigns the column names, treats '?' as missing and writes *data/heart_raw.csv* and the cleaned *data/heart_clean.csv*.

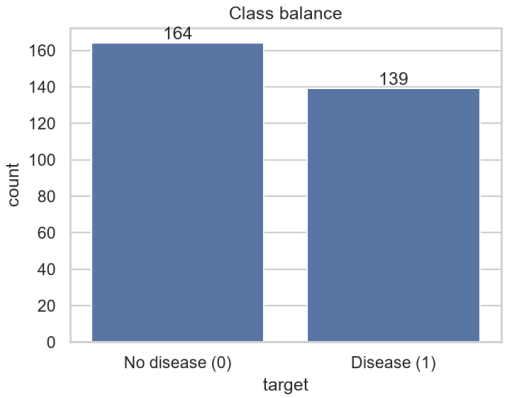
The raw data contained missing values in: *ca* (4), *thal* (2).

Missing-value handling

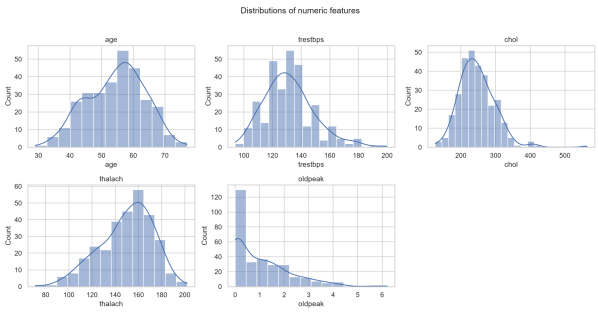
The raw data had missing values only in ca (4 rows) and thal (2 rows), about 2% of the 303 records. I state what you actually did: dropped those 6 rows, or filled them with the most frequent value. With so few rows affected the choice has little effect on the results. Give your own reason, e.g. dropping avoids inventing values, or imputing keeps all 303 rows in a small dataset. The modelling pipeline also imputes missing values itself (median for numeric, most frequent for categorical), so training does not break if gaps appear.

EDA findings

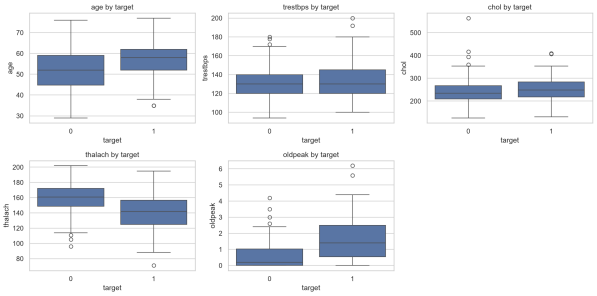
The target is fairly balanced (164 patients without disease and 139 with disease, 54.1% and 45.9%), so accuracy is a fair metric. Precision, recall and ROC-AUC are reported as well because missing a sick patient costs more than a false alarm. Skew and outliers: which numeric features (e.g. chol, oldpeak) and what you saw. Correlations: name the 2-3 features most correlated with the target and the direction. Categorical patterns: which categories clearly separate disease from no disease. What this meant for modelling, e.g. why scaling and one-hot encoding were used.



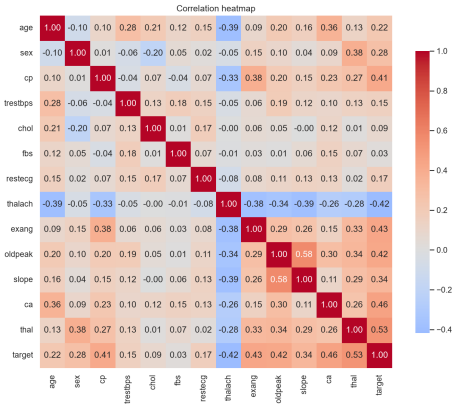
Class balance of the target.



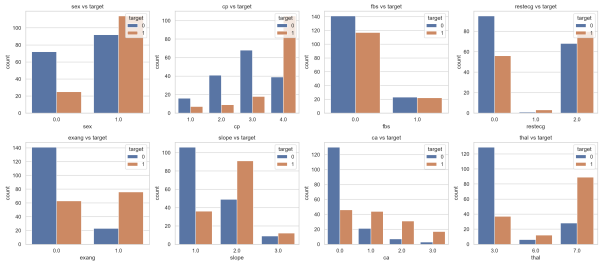
Distributions of the numeric features.



Numeric features split by target.



Correlation heatmap.



Categorical features versus the target.

5. Feature Engineering and Model Development

All preprocessing lives in one scikit-learn *ColumnTransformer* that is part of the model pipeline, so cross-validation never leaks information and the exact same transformations are applied at serving time. Numeric features (age, resting blood pressure, cholesterol, maximum heart rate, ST depression) are median-imputed and standardised. Categorical features (chest pain type, resting ECG, slope, thalassemia, number of vessels) are imputed with the most frequent value and one-hot encoded, ignoring unseen categories. Binary features (sex, fasting blood sugar, exercise-induced angina) are passed through after imputation.

Two classifiers were trained and tuned with grid search on ROC-AUC: Logistic Regression and Random Forest. Evaluation uses a stratified 80%/20% train/test split, 5-fold stratified cross-validation, and a fixed random seed (42).

Results

Model	CV acc.	CV prec.	CV recall	CV ROC-AUC	Test acc.	Test prec.	Test recall	Test ROC-AUC
random_forest	0.818	0.828	0.774	0.900	0.902	0.867	0.929	0.949
logreg	0.847	0.877	0.783	0.904	0.885	0.839	0.929	0.968

Best hyper-parameters (latest runs): **random_forest**: min_samples_leaf=3, n_estimators=300; **logreg**: C=1.

Model selection

The packaged final model is **logreg**.

Logistic Regression was selected as the final model. In cross-validation on the training split it matched or slightly beat Random Forest on every metric (ROC-AUC 0.904 vs 0.900, recall 0.783 vs 0.774), so the two are effectively tied, and the simpler model is faster, easier to explain and less prone to overfitting on about 300 rows. The hold-out test set holds only about 61 patients, so one patient changes a score by roughly

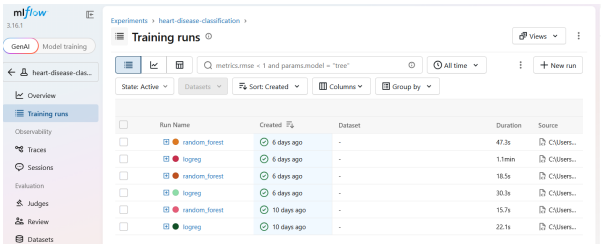
1.6 points; the test scores (ROC-AUC 0.968 vs 0.949) are too noisy to overturn the cross-validation result. For a screening use case recall matters most: at the default 0.5 threshold roughly one in five sick patients is missed in cross-validation, and lowering the decision threshold would trade precision for recall. Check these numbers against the results table above and update them if your latest runs differ.

Both models were tuned with grid search on ROC-AUC, using 5-fold stratified cross-validation on the training split only: Logistic Regression over the regularisation strength C, and Random Forest over the number of trees, the maximum depth and the minimum samples per leaf. The hold-out set was kept untouched so that it gives an unbiased final check. Add one observation of your own, e.g. how much tuning changed the scores or what else you tried.

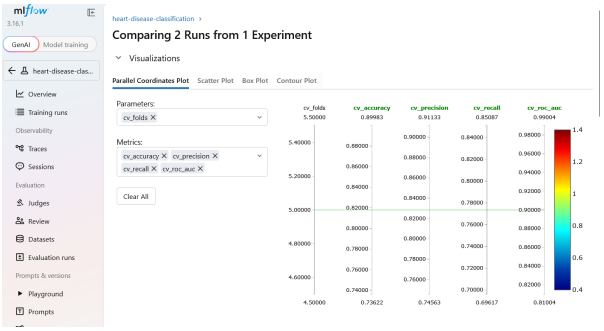
6. Experiment Tracking with MLflow

Every training run logs its parameters (model type, seed, folds, tuned hyper-parameters), cross-validation and hold-out metrics, the ROC curve and confusion matrix as plots, and the fitted pipeline with an input example and signature. Each hyper-parameter combination of the grid search is a nested child run, and the final model is tagged and registered in the MLflow model registry. The tracking database holds 6 training runs and 48 nested grid-search runs in the experiment *heart-disease-classification*.

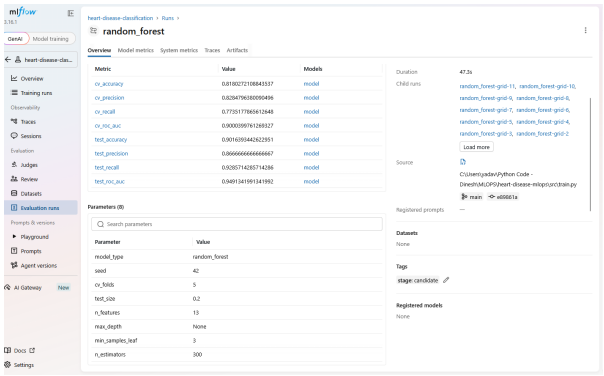
Runs were compared in the MLflow UI by cross-validation ROC-AUC and recall, and the nested grid-search runs show how performance changed with each hyper-parameter setting. I re-ran training with what you changed: a different seed, a different grid, ... to check that the results were stable. Add what you learned from the comparison view.



MLflow experiment page listing the tracked runs.



MLflow comparison of runs (parameters and metrics side by side).



A single MLflow run: parameters, metrics and logged artifacts (ROC curve, confusion matrix, model).

7. Model Packaging and Reproducibility

The final pipeline (preprocessing plus model) is saved with joblib as *models/final_model.joblib*, next to *models/model_metadata.json*, which records the model name, MLflow run id, hyper-parameters, feature list, metrics and library versions. The same model is also stored as an MLflow model. Dependencies are pinned in *requirements.txt* (full project) and *requirements-api.txt* (serving image only).

Versions recorded at training time: python 3.11.9, scikit-learn 1.9.1, mlflow 3.16.1.

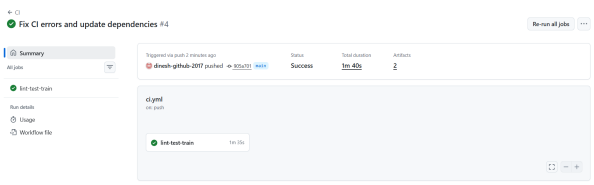
To confirm that a clean setup works, I created a fresh virtual environment, or cloned the repository into a new folder, installed requirements.txt, and ran python -m src.train and python -m pytest, with state the result. The GitHub Actions pipeline repeats the same steps from scratch on every push using Python 3.11 and passed, which is an independent check of the requirements file.

8. CI/CD Pipeline and Automated Testing

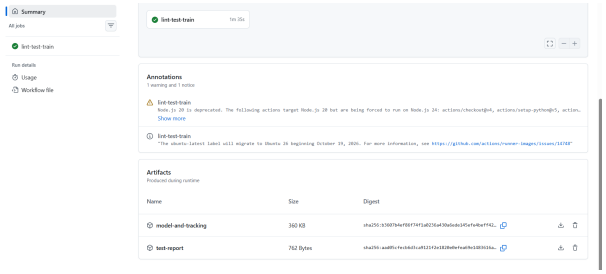
The test suite contains 29 pytest tests covering data cleaning, the preprocessing pipeline, model training helpers, the prediction function and the API (valid requests, missing fields, out-of-range values, wrong types and metrics). The GitHub Actions workflow (*.github/workflows/ci.yml*) runs on every push and pull request:

Step	Purpose
Install dependencies	Clean environment from the requirements files
flake8	Linting of src, api and tests
pytest	Unit tests; JUnit report uploaded as an artifact
Train model	Runs the training script; model and MLflow data uploaded as artifacts
Docker build	Builds the serving image
Container smoke test	Starts the container, checks /health and /predict

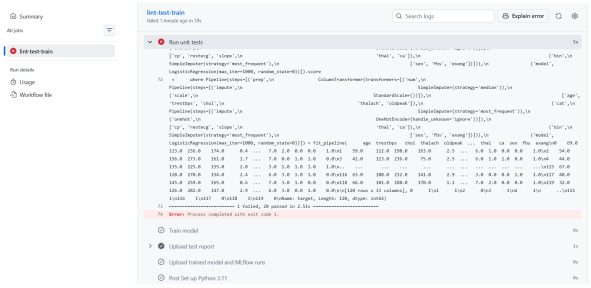
The pipeline fails on lint, test or build errors and prints clear logs, demonstrated below with a deliberately failing test.



GitHub Actions run with every step passing (lint, tests, training, Docker build, container smoke test).



Artifacts produced by the workflow run.



Deliberately failing run: the pipeline stops on a failing test and shows a clear error log.

9. Model Containerisation and API

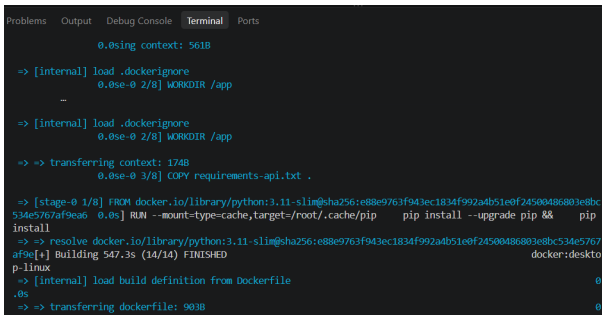
The model is served by a FastAPI application. Inputs are validated with allowed ranges, so invalid requests are rejected with HTTP 422. The image is based on python:3.11-slim, contains only the serving dependencies, runs as a non-root user and has a health check. Request logs record method, path, status and latency, and the prediction outcome, but never the patient's health data.

Endpoint	Description
POST /predict	JSON patient record in; prediction, label, confidence and risk probability out
GET /health	Readiness/liveness; confirms the model is loaded
GET /metrics	Prometheus metrics
GET /model-info	Packaged model metadata

Example request

```
{
  "age": 54,
  "sex": 1,
  "cp": 3,
  "trestbps": 130,
  "chol": 246,
  "fbs": 0,
  "restecg": 0,
  "thalach": 150,
  "exang": 0,
  "oldpeak": 1.0,
  "slope": 2,
  "ca": 0,
  "thal": 3
}
```

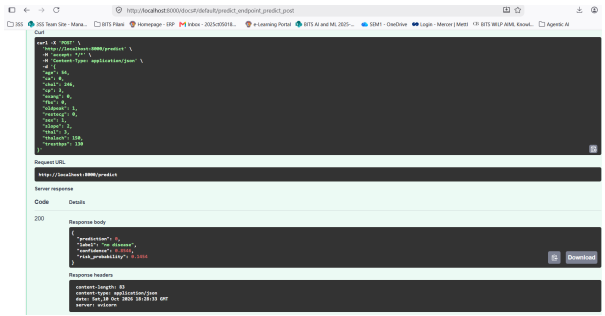
confidence is the probability of the predicted class; *risk_probability* is the probability of heart disease.



Docker image build completing successfully.

Screenshot missing: screenshots/docker_run.png

Container running locally with request log lines.



Successful call to /predict returning prediction and confidence.

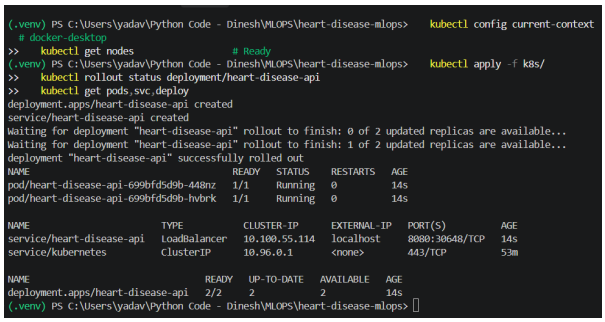


Interactive API documentation (Swagger UI) at /docs.

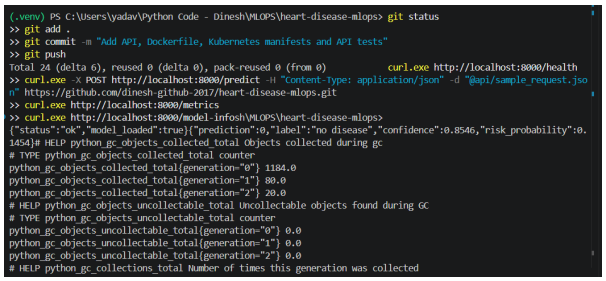
10. Production Deployment on Kubernetes

The image is deployed to a local Kubernetes cluster (Docker Desktop) with a Deployment of two replicas, readiness and liveness probes on /health, and resource requests and limits. A Service of type LoadBalancer exposes it. Access: http://localhost:8080 (local Kubernetes, Docker Desktop); see Section 8.

The complete manifests are in the k8s/ folder of the repository: *deployment.yaml* (2 replicas, probes, resource limits, Prometheus scrape annotations) and *service.yaml* (LoadBalancer, port 8080 to container port 8000).



kubectl output: Deployment, 2 pods Running (1/1) and the LoadBalancer Service.



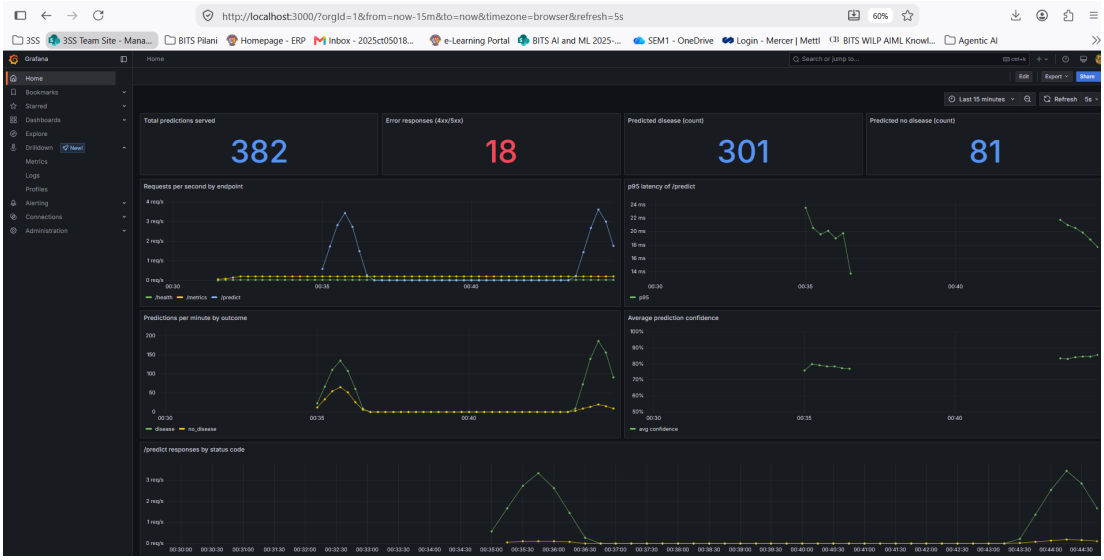
Health check and prediction through the Kubernetes service.

11. Monitoring and Logging

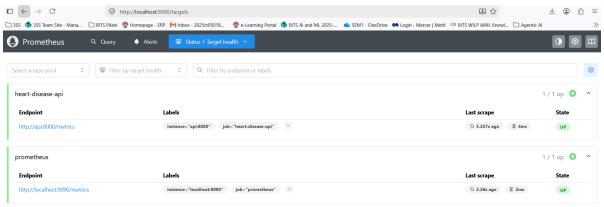
The API exposes Prometheus metrics, scraped every 5 seconds. Docker Compose runs the API, Prometheus and Grafana together; Grafana loads a provisioned dashboard. A single API container is monitored so that counters are consistent (with several pods behind one service, each scrape would hit a different pod).

Metric	Meaning
http_requests_total	Requests by method, path and status code
http_request_duration_seconds	Latency histogram per path
predictions_total	Predictions by outcome (disease / no_disease)
prediction_confidence	Histogram of prediction confidence

With normal traffic (200 requests from the traffic script) the dashboard showed 193 successful predictions and 7 rejected requests, matching the 200 sent; the 7 are deliberately invalid requests that the API rejected with HTTP 422. The p95 latency of /predict stayed at roughly 14 to 24 ms. The test traffic is randomly generated, so the outcome mix reflects the generator, not real patients. Drift batch: describe what changed in the outcome mix and the average confidence when you ran --drift, and what you conclude. This is a simple illustration that a shift in incoming data shows up in prediction behaviour; it is not formal drift detection, which would compare feature distributions against the training data.



Grafana dashboard with request, latency, prediction and confidence panels.



Prometheus targets page: API target is UP.

Screenshot missing: screenshots/compose_logs.png

API request logs (docker compose logs api).

12. Repository, Deliverables and Limitations

Code repository: https://github.com/dinesh-github-2017/heart-disease-mlops

Deliverable	Status
Code (src/, api/)	Present
Dockerfile	Present
requirements.txt	Present
Cleaned dataset	Present
Download script	Present
Notebooks	Present
Unit tests	Present
CI workflow	Present
Kubernetes manifests	Present
Monitoring configuration	Present
Screenshots folder	Present

Limitations and future work

1. The dataset is small (about 300 patients from one study), so metrics are noisy and may not generalise; no real patient data was used and the model is a learning exercise, not a medical device. 2. The decision threshold is fixed at 0.5; a screening deployment would tune it for recall. 3. Kubernetes ran on a local single-node cluster using the local Docker image store, so there is no real high availability, autoscaling, cloud load balancer or container registry. 4. Monitoring covers service health and prediction behaviour only; there is no feature-level drift detection, alerting or automated retraining. Add your own next steps, e.g. a registry and cloud deployment, threshold tuning, drift alerts, automated retraining.